Minimum Swaps Approach

The key observation is: **if there are `k` ones in the array, then after grouping them together, they must occupy exactly `k` consecutive positions.**

So don't think about *which elements to swap* first. Think:

> **Which circular window of size `k` is the best place to put all the `1`s?**

Hint 1: Count the total `1`s

```
nums = [0,1,0,1,1,0,0]

totalOnes = 3
```

Since there are 3 ones, we need to find **3 consecutive positions** where grouping them requires the fewest swaps.

Hint 2: Look at every window of size `totalOnes`

For example:

```
[0,1,0]  → 1 one, 2 zeros
[1,0,1]  → 2 ones, 1 zero
[0,1,1]  → 2 ones, 1 zero
[1,1,0]  → 2 ones, 1 zero
...
```

Suppose a window of size `k` contains:

```
ones = 2
zeros = 1
```

That means:

```
[1, 0, 1]
```

To make this window contain all `1`s, you only need to bring **one `1` from outside** and swap it with that `0`.

So:

> **Number of swaps needed = number of zeros inside the window**

And:

```
zeros = windowSize - numberOfOnes
```

Therefore, you want the window containing the **maximum number of `1`s**.

Hint 3: Why circular?

A valid group can cross the boundary:

```
index:  0 1 2 3 4
nums = [1,1,0,0,1]
```

The three `1`s are actually together circularly:

```
        ┌──────────────┐
        ↓              │
[1, 1, 0, 0, 1]        │
 ↑                     ↑
 └──── adjacent ───────┘
```

So one simple trick is to conceptually make the array circular by checking beyond the end:

```
C++
nums[i % n]
```

Then use a **sliding window of size `totalOnes`**.

Your approach should be:

1. Count total number of `1`s → `k`
2. Use a sliding window of size `k`
3. Make the window circular using `% n`
4. Track how many `1`s are inside each window

5. Find the window with the maximum `1`s
6. Answer:

```
minimum swaps = k - maximum ones in any window
```

💡 **Main intuition:**
Instead of asking **"How do I swap all the ones?"**, ask:

> **"Among all possible circular blocks where the ones could end up, which block already contains the most ones?"**

That block needs the fewest replacements/swaps.

Sure. Here is the **C++ sliding window solution**:

```
C++
class Solution {
public:
    int minSwaps(vector<int>& nums) {
        int n = nums.size();

        // Count total number of 1's
        int totalOnes = 0;

        for (int num : nums) {
            totalOnes += num;
        }

        // If there are 0 or 1 ones, they are already grouped
        if (totalOnes <= 1) {
            return 0;
        }

        int onesInWindow = 0;
        int maxOnes = 0;

        // Sliding window of size = totalOnes
        // Go up to n + totalOnes - 1 to handle circular array
        for (int i = 0; i < n + totalOnes - 1; i++) {

            // Add current element using modulo for circular behavior
            if (nums[i % n] == 1) {
                onesInWindow++;
            }

            // Remove element that goes outside the window
            if (i >= totalOnes) {
                if (nums[(i - totalOnes) % n] == 1) {
                    onesInWindow--;
                }
            }

            // Window is valid when its size becomes totalOnes
            if (i >= totalOnes - 1) {
                maxOnes = max(maxOnes, onesInWindow);
            }
        }

        // Zeros inside the best window = swaps required
        return totalOnes - maxOnes;
    }
};
```

Main logic

```
Window size = total number of 1's

Find the circular window with:
maximum number of 1's

Answer =
window size - maximum 1's
```

For example:

```
nums = [0,1,0,1,1,0,0]

totalOnes = 3

Possible best window:

[1,0,1]

ones = 2
zeros = 1

Minimum swaps = 3 - 2 = 1
```

Complexity

- **Time:** `O(n)`
- **Space:** `O(1)`

The important part is:

```
C++
nums[i % n]
```

This makes the array behave like a **circular array** without actually creating a second array.

Sure. Let's do a **complete dry run** of the code using:

```
C++
nums = [0,1,0,1,1,0,0]
```

1. Count total 1s

```
nums = [0,1,0,1,1,0,0]
        ↑   ↑ ↑

totalOnes = 3
```

So our sliding window size will be:

```
windowSize = 3
```

We need to find a **circular window of size 3** containing the maximum number of 1s.

---

2. Variables initially

```
C++
onesInWindow = 0;
maxOnes = 0;
n = 7;
totalOnes = 3;
```

The loop runs until:

```
C++
i < n + totalOnes - 1
```

So:

```
i < 7 + 3 - 1
i < 9
```

Therefore:

```
i = 0 to 8
```

The extra elements simulate the circular array.

Conceptually:

```
Original:
[0, 1, 0, 1, 1, 0, 0]

Circular extension:
[0, 1, 0, 1, 1, 0, 0, 0, 1]
                     ↑  ↑
                  repeated
```

Because:

```
C++
nums[i % n]
```

Dry Run 🔥

i = 0

```
C++
nums[0 % 7] = nums[0] = 0
```

Add `0` → `onesInWindow` remains `0`.

```
Window: [0]
onesInWindow = 0
```

Window size is not yet 3.

---

i = 1

```
C++
nums[1 % 7] = nums[1] = 1
```

Add it:

```
Window: [0,1]

onesInWindow = 1
```

Still window size < 3.

---

i = 2

```
C++
nums[2 % 7] = nums[2] = 0
```

```
Window: [0,1,0]

onesInWindow = 1
```

Now the window size becomes 3.

```
C++
maxOnes = max(0, 1) = 1
```

State:

```
Window = [0,1,0]
maxOnes = 1
```

i = 3

First add:

```
C++
nums[3] = 1
```

Temporarily:

```
[0,1,0,1]
onesInWindow = 2
```

But window size became 4, so remove:

```
C++
i - totalOnes = 3 - 3 = 0
```

Remove:

```
C++
nums[0] = 0
```

Since it is `0`, `onesInWindow` stays:

```
Window = [1,0,1]

onesInWindow = 2
```

Update:

```
C++
maxOnes = max(1, 2) = 2
```

i = 4

Add:

```
C++
nums[4] = 1
```

Temporary:

```
[1,0,1,1]

onesInWindow = 3
```

Now remove the element outside the window:

```
C++
i - totalOnes = 4 - 3 = 1
```

Remove:

```
C++
nums[1] = 1
```

So:

```
onesInWindow = 2
```

Final window:

```
[0,1,1]
```

Update:

```
maxOnes = max(2,2) = 2
```

i = 5

Add:

```
C++
nums[5] = 0
```

Window temporarily:

```
[0,1,1,0]
```

Remove index:

```
5 - 3 = 2
```

Remove:

```
nums[2] = 0
```

So:

```
Window = [1,1,0]
onesInWindow = 2
maxOnes = 2
```

i = 6

Add:

```
C++
nums[6] = 0
```

Temporary:

```
[1,1,0,0]
```

Remove index:

```
6 - 3 = 3
```

Remove:

```
nums[3] = 1
```

Therefore:

```
onesInWindow = 1
```

Final:

```
Window = [1,0,0]

maxOnes = 2
```

Now the circular part starts 🔄

`i = 7`

```
C++
i % n = 7 % 7 = 0
```

So we go back to:

```
nums[0] = 0
```

Add `0`.

Temporary:

```
[1,0,0,0]
```

Remove:

```
C++
i - totalOnes = 7 - 3 = 4
```

```
nums[4] = 1
```

So:

```
Window = [0,0,0]

onesInWindow = 0
maxOnes = 2
```

This window crosses the boundary:

```
Original:
[0,1,0,1,1,0,0]
               └─────┘
                  ↓

Circular window:
[0,0,0]
 ↑     ↑
index 5,6,0
```

`i = 8`

```
C++
i % n = 8 % 7 = 1
```

So:

```
nums[1] = 1
```

Add:

```
[0,0,0,1]
onesInWindow = 1
```

Remove index:

```
C++
8 - 3 = 5
```

```
nums[5] = 0
```

So no change.

Final window:

```
[0,0,1]
onesInWindow = 1
maxOnes = 2
```

Final result

We found:

```
totalOnes = 3
maxOnes in any circular window = 2
```

Therefore:

```
minimum swaps = totalOnes - maxOnes
              = 3 - 2
              = 1
```

Answer

```
1
```

The most important thing to understand

At every point, our window represents the **place where we want all 3 ones to be grouped**.

For example:

```
[1,0,1]
```

This window already has:

```
2 ones
1 zero
```

We need all 3 positions to become `1`:

```
[1,0,1]  →  [1,1,1]
```

So we only need to swap **one outside** `1` with the `0` inside.

That's why:

```
swaps = number of zeros inside the best window
      = totalOnes - maximumOnesInWindow
      = 1
```

The sliding window is simply checking **every possible circular group of size** `totalOnes` and choosing the one that already contains the most `1`s.